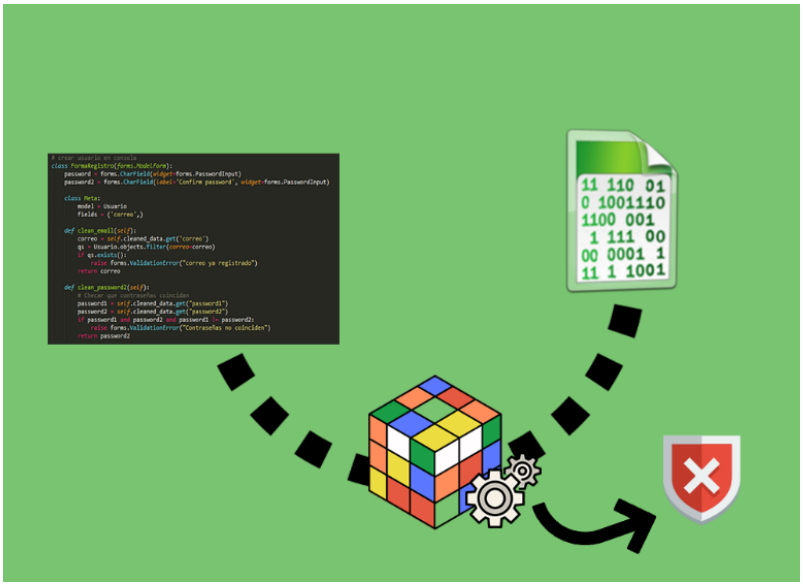
AUTÓMATAS Y COMPILADORES

UNIDAD

3



EXPRESIONES REGULARES Y GRAMATICAS

© **Se prohíbe la reproducción total o parcial de esta obra, por cualquier medio, electrónico o mecánico sin la autorización de la Dirección de Educación a Distancia (UDB Virtual).**

Dirección de Educación a Distancia

Apartado Postal, 1874, San Salvador, El Salvador

Tel: 2251-8200 ext: 1743

©

Sumario

Introducción a la Unidad 3 1
Clase 9| Lenguajes regulares y no regulares 2
Clase 10| Expresiones regulares 10
Clase 11| Gramáticas regulares 21
Clase 12| Autómatas, complejidad y computabilidad 28
Referencias citadas en UNIDAD 3 35
Glosario de los términos citados en la UNIDAD 3 36

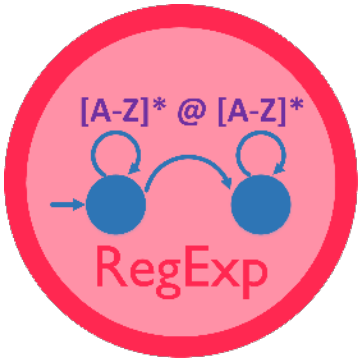
Introducción a la Unidad 3

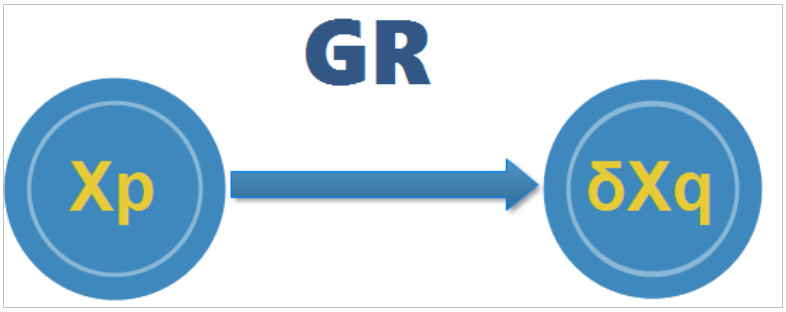
En la unidad 2 aprendimos a construir autómatas finitos deterministas y no deterministas, aplicando reglas de equivalencia y simplificación.

En esta unidad 3 se considera la teoría de los lenguajes formales, expresiones regulares, gramática regular y autómatas, los cuales nos ayudan a procesar, compilar e interpretar lenguajes de programación. Utilizaremos las competencias adquiridas sobre los autómatas finitos vistos en la unidad 2 para encontrar las equivalencias y transformaciones entre los autómatas finitos, las expresiones regulares y gramáticas.

Además, se desarrollará la construcción de autómatas e identificación de los lenguajes de programación. Considerando que un autómata es un sistema que toma información, la transforma y produce otra información, información que es transmitida al medio ambiente, tomando en cuenta las teorías de complejidad y de la computabilidad.

Al finalizar esta unidad el estudiante lograra adquirir las competencias a través de ejemplos y ejercicios que desarrollaremos a lo largo de la unidad.





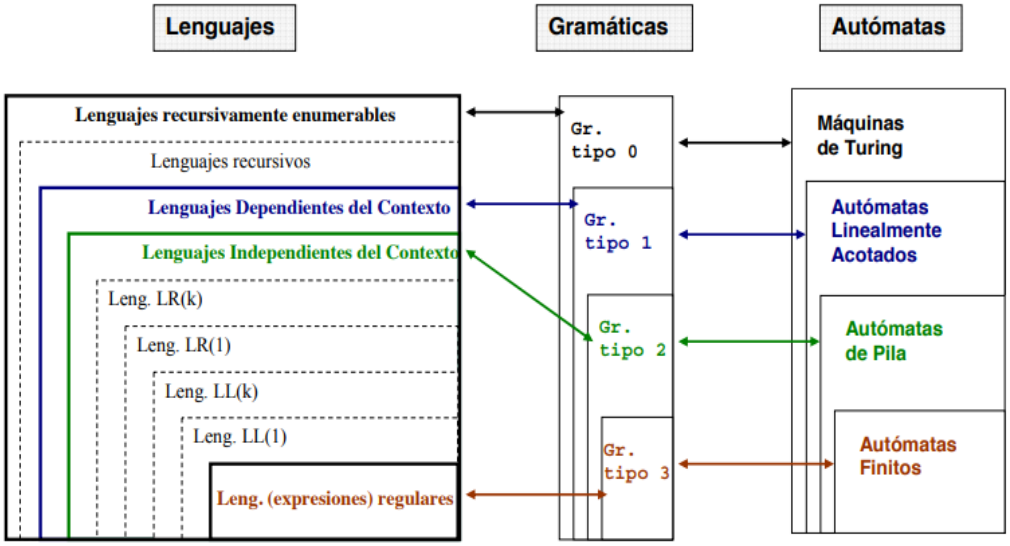
Clase 9| Lenguajes regulares y no regulares

9. Lenguajes regulares

Chomsky clasificó las gramáticas formales y los lenguajes que estas generan de acuerdo con una jerarquía de cuatro niveles representada en Figura 1. Es posible establecer una relación biunívoca entre los diferentes niveles de la jerarquía de Chomsky y cuatro niveles de una jerarquía definida entre los diferentes tipos de autómatas. A cada nivel de gramática se le puede asociar de forma natural un conjunto de lenguajes que serán los que esas gramáticas generan, además, se le puede asociar una clase de autómatas formada por aquellos que podrán reconocer a dichos lenguajes.

Figura 1

Jerarquía de gran áticas, lenguajes y autómatas



Fuente: *Nota.* Por Jurado (2008)

Cada nivel de lenguaje se corresponde con un tipo de autómata. Por ejemplo, nosotros hemos estudiado los Autómatas Finitos, y estos corresponden a un lenguaje de tipo 3, los cuales son los lenguajes regulares que pertenece a una gramática regular. Los autómatas finitos serían los más básicos, es decir, si queremos utilizar un lenguaje de tipo 2 será necesario utilizar un autómata más complejo.



Concepto

Para Hernández (2010) “Los lenguajes regulares son aquellos que pueden describirse mediante un autómata finito” (p. 31).



Concepto

Los lenguajes regulares son aquellos lenguajes cuyas cadenas está formadas por la concatenación de símbolos, en las cuales no hay relación entre una parte de la cadena y otra parte de la cadena. (Amaya, 2015).

Los lenguajes regulares son caracterizados por contener en sus palabras *regularidades*, es decir, repeticiones de los mismos elementos, como por ejemplo en el siguiente lenguaje: *L = {ac, acac, acacac, acacacac, ...}*

Se puede apreciar en el ejemplo que las palabras de *L* son repeticiones de *“ac”* en cualquier número de veces. Entonces la *regularidad* consiste en que las palabras contienen *“ac”* algún número de veces.

Veamos otro ejemplo, sea el siguiente lenguaje regular:

L1 = {abc, cc, abab, abccc, ababc, ...}

La regularidad en L1 consiste en que sus palabras comienzan con repeticiones de *“ab”*, seguidas de repeticiones de *“c”*. Similarmente es posible definir muchos otros lenguajes basados en la idea de repetir esquemas simples. Esta es la idea básica para formar los lenguajes regulares.

9.1 Definición formal de los lenguajes regulares

Los lenguajes regulares son lenguajes formales que tienen estas características:

- Puede ser *descrito* mediante una expresión regular, de manera compacta expresando las cadenas de símbolos que le pertenecen.
- Puede ser *generado* mediante una gramática regular, obteniendo todas las cadenas de símbolos que le pertenecen.
- Puede ser *reconocido* mediante un autómata finito, al evaluar si una cadena de símbolos pertenece a él o no.

Un lenguaje *L* es regular sí y sólo si se cumple al menos una de las reglas siguientes:

- *L* es finito.
- *L* es la unión o la concatenación de otros lenguajes regulares *L1* y *L2*, $L3 = L1 \cup L2$, $L3 = L1L2$.
- *L* es la cerradura de Kleene de algún lenguaje regular, $L = R^*$.

En la Tabla 1 se muestran los ejemplos de las reglas anteriores sobre $\Sigma = \{a,b\}$.

Tabla 1

Ejemplos de las reglas de los lenguajes regulares

Regla	Ejemplo
Vacío y el lenguaje conformado por la palabra vacía son lenguajes regulares.	a) ϕ b) $\{\varepsilon\}$
Los lenguajes formados por una sola palabra de un solo símbolo son regulares	c) {a} d) {b}
Los lenguajes que resultan de unir dos lenguajes regulares son regulares.	e) {a, b} que es la unión de c) y d) f) { ε, b} que es la unión de b) y d) g) { ε, a, b} que es la unión de b) y e)

Los lenguajes que resultan de concatenar dos lenguajes regulares son regulares.	h) {a, aa, ba} que es la concatenación de g) y c) i) {a, ab, b, bb} que es la concatenación de e) y f)
Los lenguajes que resultan de realizar una cerradura de estrella a un lenguaje regular son regulares.	j) { ε, a, a^2, a^3, ...} que es la cerradura de estrella de c) k) { ε, b, b^2, b^3, ...} que es la cerradura de estrella de d)

Fuente: *Nota.* Elaboración propia.

9.2 Demostración si un lenguaje es regular

Para probar que un determinado lenguaje es regular en un alfabeto dado, tenemos dos procedimientos o métodos.

1. Podemos enumerar los lenguajes regulares sobre el alfabeto hasta que se encuentre el lenguaje dado.
2. Podemos expresar el lenguaje en términos de uniones, uniones o concatenaciones o cerraduras de estrellas de lenguajes unitarios.

Para los siguientes dos ejemplos usaremos el método 2, por ser más práctico.

Ejemplo 1. Demostraremos que $A = \{ \varepsilon, bb, a\}$ sobre $\{a, b\}$ es un lenguaje regular.

$$\{ \varepsilon, bb, a\} = \{\varepsilon\} \cup \{bb\} \cup \{a\} = \{\varepsilon\} \cup \{b\}\{b\} \cup \{a\}$$

Esta última expresión $\{\varepsilon\} \cup \{b\}\{b\} \cup \{a\}$ solo tiene operaciones permitidas de unión y concatenación. Además, todos los lenguajes que componen esta expresión son unitarios. Por tanto, el lenguaje dado es un *lenguaje regular*.

Ejemplo 2. Demostraremos que $A = \{cd(ab)^i \mid i \geq 0\}$ es un lenguaje regular.

$$\{cd(ab)^i \mid i \geq 0\} = \{cd(ab)^0, cd(ab)^1, cd(ab)^2, cd(ab)^3, \ldots\}$$

$$= \{cd\} \{(ab)^0, (ab)^1, (ab)^2, (ab)^3, \ldots\}$$

$= \{c\}\ \{d\}\ \{ab\}^*$

$= \{c\}\ \{d\}\ \{a\}\ \{b\}^*$

La expresión obtenida $\{c\}\ \{d\}\ \{a\}\ \{b\}^*$ está conformada de conjuntos unitarios, con operaciones de uniones o concatenaciones y cerraduras de estrellas. Por lo tanto, se demuestra que el lenguaje dado también es un *lenguaje regular*

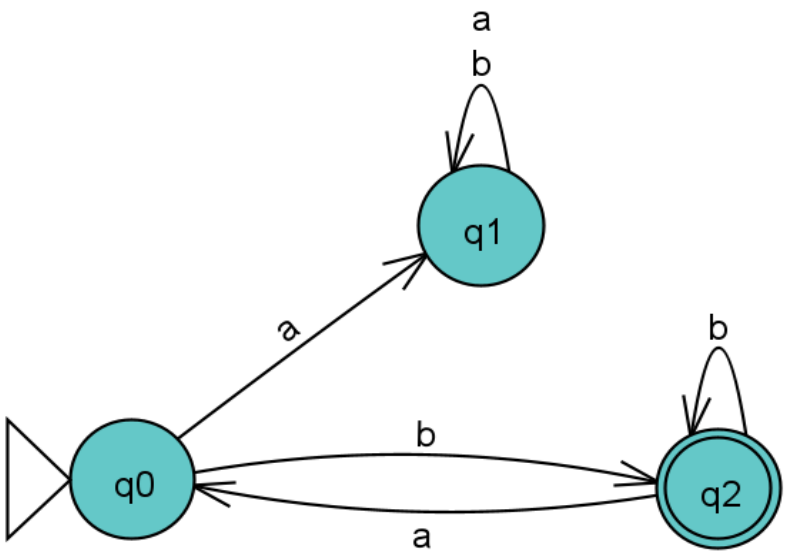
9.3 Limitaciones de los lenguajes regulares

Un autómata finito se limita a los estados para recordar la secuencia de símbolos recibidos en un momento dado. En otras palabras, cabe señalar que, en el caso de un AF, el único rastro de un símbolo detectado es el estado en el que se encuentra. Por la misma razón, varias secuencias diferentes de caracteres que conducen al mismo estado se consideran *indistinguibles*.

Por ejemplo, para el AFD de la Figura 2, las secuencias de caracteres *bab* y *bbbbb* son indistinguibles, pues ambas llevan al estado q2.

Figura 2

Estados indistinguibles: estado q2



Fuente: *Nota.* Elaboración propia.

Concepto

Para Hopcroft et al. (2008) El lema de bombeo establece que, si un lenguaje es regular, entonces toda cadena lo suficientemente larga del lenguaje tiene una subcadena no vacía que puede ser "bombeada", es decir, repetida cualquier número de veces siempre y cuando las cadenas resultantes pertenezcan también al lenguaje. Este hecho puede utilizarse para demostrar que muchos lenguajes no son regulares. (p.139)

Esta limitación de los AF los hace finalmente incapaces de distinguir las palabras aceptables de las no aceptables en ciertos lenguajes, más complicados que los lenguajes regulares. Por ejemplo, para el lenguaje $\{a^n\ b^n\}$ no es posible construir un autómata finito que lo acepte, ni representarlo por una expresión o gramática regular. En efecto, supongamos que un AFD está recorriendo una palabra $a^n\ b^n$, entonces al terminar el grupo de *a's* el autómata debe recordar cuántas encontró, para poder comparar con el número de *b's*. Ahora bien, como la cantidad de *a's* que puede haber en la primera mitad de la palabra es arbitraria, dicha cantidad no puede recordarse con una cantidad de memoria fija, como es la de los autómatas finitos.

Formalmente, estableceremos un teorema que nos permite demostrar los límites de los autómatas finitos. Si L es un lenguaje regular infinito, entonces existen cadenas x, y, z tales que $y \neq \varepsilon$, y $xy^n z \in L$, para algún $n \geq 0$.

Este resultado muestra que, asumiendo que el lenguaje es regular, el lenguaje debe contener palabras en las que las subcadenas se repiten un número arbitrario de veces. Es decir, podemos insertar repetidamente *"bombear"* subcadenas en ciertas palabras del lenguaje sin que el autómata lo detecte. Este caso permite probar por contradicción que un determinado lenguaje no es regular.

Ejemplo: Como un ejemplo de la aplicación de este teorema, probaremos que el lenguaje $\{a^n\ b^n\}$ no es regular.

Si asumimos que es regular. Entonces, por el teorema de bombeo, debe haber palabras de la forma *xyz*, comenzando en una cierta longitud, donde la parte *"y"* se puede repetir varias veces. Existen 3 posibilidades:

1. Qué *"y"* no contenga caracteres distintos a *"a"*, es decir, *y = aa...a*. En este caso, al repetir varias veces *"y"*, habrá más *a's* que *b's* y la palabra no tendrá la forma deseada. Es decir, suponiendo que $\{a^n\ b^n\}$ es regular hemos llegado a la conclusión de que contiene palabras con más *a's* que *b's*, lo cual es una contradicción.
2. Qué *"y"* no contenga caracteres distintos de "*b"*. Este caso es similar al caso (1).
3. Qué *"y"* contenga *a's* y *b's*, es decir, *y = aa...abb...b*. Pero en este caso, al repetirse *"y"*, las *a's* y *b's* quedarán en desorden en la palabra, la cual no tendrá la forma $\{a^n\ b^n\}$. También en este caso hay contradicción.

Por lo tanto, el suponer que $\{a^n\ b^n\}$ es regular nos lleva a contradicción. Se concluye que $\{a^n\ b^n\}$ no es regular

Es muy importante notar que para las pruebas por contradicción usando el teorema de bombeo hay que explorar exhaustivamente todas las posibles maneras de dividir la palabra *"w"* en *xyz*, y encontrar contradicción en cada posible división. Con una sola división posible en que no se encuentre una contradicción, la prueba fracasa. Al fracasar la prueba, no se puede concluir ni que el lenguaje es regular ni que no lo es; simplemente no se llega a ninguna conclusión.

Otros lenguajes que tampoco son regulares son:

- *{ww}*, que es el lenguaje cuyas palabras tienen dos mitades iguales.
- $\{ww^R\}$, que es el lenguaje cuyas palabras tienen dos mitades simétricas.
- El lenguaje de las palabras palíndromas, que se leen igual al derecho y al revés, como por ejemplo *ANITALAVALATINA*,
- El lenguaje de los paréntesis bien balanceados, como *()(()), ()()(), ((()))*, etc.



Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender el lema de bombeo para determinar si los lenguajes son regulares:

Juancar Molinero. (24 de abril de 2013). *El Lema de Bombeo y los Lenguajes Regulares.* [Archivo de Vídeo]. Youtube. https://youtu.be/-1KbhcKzEuM

9.4 Lenguajes Libres de Contexto

Según la jerarquía de Chomsky, los lenguajes libres de contexto (LLC) forman una categoría de lenguajes más amplia que los lenguajes regulares. Estos lenguajes son importantes tanto desde el punto de vista teórico como práctico porque casi todos los lenguajes de programación están basados en LLC. A partir de la década de 1970, con lenguajes como Pascal, se convirtió en una práctica común formalizar gramáticas de lenguajes de programación utilizando herramientas basadas en las Gramáticas Libres de Contexto, que representan a los LLC. Por otro lado, el análisis automático de LLC es computacionalmente más eficiente que otras clases de lenguaje más generales.



Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender que son los lenguajes libres de contexto y la gramática que los genera:

Cesar Perez. (9 de mayo de 2020). *Lenguajes Libres de Contexto.* [Archivo de Vídeo]. Youtube. https://youtu.be/YCKXDclmv00

Clase 10| Expresiones regulares

10.Expresiones regulares

Concepto

"Una expresión regular (ER) es una notación normalizada para representar lenguajes regulares, es decir, lenguajes generados por gramáticas de tipo 3" (Amaya, 2015, p. 49).

La notación de conjuntos nos permite describir lenguajes regulares, pero necesitamos una representación donde la representación del lenguaje sea solo texto (cadenas). Entonces, la representación de un lenguaje regular será solo palabras de un lenguaje (representación correctamente formada). Con estas ideas definiremos un lenguaje, una expresión regular, donde cada palabra representará un lenguaje regular.

10.1 Definición de las expresiones regulares

Las expresiones regulares (RE) le permiten describir con precisión y sencillez cualquier lenguaje regular. Se deben utilizar todos los símbolos del alfabéticos Σ así como λ y ϕ para definir una ER.

Sea Σ un alfabeto. El conjunto ER de las expresiones regulares sobre Σ contiene las cadenas en el alfabeto $\Sigma \cup \{$"λ", "+", "•", "∗", "(", ")", "Φ"$\}$ que cumplen con lo siguiente:

1. *"λ" y "Φ" $\in$ ER*
2. Si $\sigma \in \Sigma$, entonces $\sigma \in ER$.
3. Si *E1*, *E2* $\in$ *ER*, entonces *"(" E1"+" E2")" $\in$ ER, "(" E1"•" E2")" $\in$ ER, "(" E1") *" $\in$ ER*.

En la Tabla 2 se presentan algunos operadores utilizados en las *ER*

Tabla 2

Operadores utilizados en las ER

Operador	Función
+	Representa la unión.
•	Representa concatenación, este operador no se suele escribir.
*****	Representa el cierre de Kleene.
()	Modifican las propiedades de los demás operadores.

Fuente: *Nota.* Elaboración propia.

Ejemplos. Son ER en {a, b, c} las siguientes;

- “a”
- “((a + b)) *”
- “((a • b) • c)”

No son ER:

- “ab”
- “((a • b (c)*)”

El orden de prioridad de los operadores es, de mayor a menor: * , ▪ , +

Este orden puede alterarse mediante paréntesis, de forma análoga a como se hace con las expresiones aritméticas.

10.2 Significado de las expresiones regulares

Las ER son fórmulas que tienen como objetivo representar el lenguaje. Por lo tanto, el significado de la ER es el lenguaje que representa. Por ejemplo, la ER *"Φ"* representa el conjunto vacío { }.

Para obtener una comprensión intuitiva de cómo un ER representa un lenguaje, veamos el proceso de verificar si una palabra dada *“w”* pertenece al lenguaje representado por un

ER dada. Diríamos que una palabra "*coincide*" con una expresión regular si es parte del lenguaje que representa.

La palabra vacía ε "*coincide*" con la ER λ.

Una palabra de una letra como *"a" coincide* con una ER consistente en la misma letra *"a"*, *"b"* coincide *"b"*, etc.

Una palabra $w = uv$, es decir w consta de dos partes u y v, *coincide* con una expresión $(U \bullet V)$ siempre que *u coincida* a U y *v coincida* a V.

Por ejemplo, *abc coincide* con $(a \bullet (b \bullet c))$ porque *abc* puede ser dividida en *a* y *bc*, y *a coincide* con *a* en la ER, mientras que *bc coincida* con $(b \bullet c)$ separando *b* y *c* de la misma manera.

De manera similar, si la ER tiene la forma $(U + V)$, puede *coincidir* con *w* cuando *coincide* con U o V. Por ejemplo, *bc coincide* con (a + (b • c)). Una palabra *"w" coincide* a una expresión U^* si *"w"* se puede dividir en fragmentos $w = w1w2, ...$ tal que cada fragmento de w_i corresponde a U.

Por ejemplo, *caba coincide* con $(((c + b) \bullet a))^*$ porque puede partirse en los pedazos *ca* y *ba*, y ambos *coincide* con $((c + b) \bullet a)$, lo cual es fácil de verificar. A continuación, definiremos formalmente la correspondencia entre la representación (una ER) y el lenguaje representado.

El significado de una ER es la función $L: ER \rightarrow 2^{\Sigma^*}$, es decir, una función que toma como entrada una expresión regular y como salida un lenguaje. La función se define de la manera siguiente:

1. L ("Φ") = $\emptyset$ (el conjunto vacío)
2. L ("Λ") = $\{\varepsilon\}$
3. L ("σ") = $\{\sigma\}$, $\sigma \in \Sigma$.
4. L ("("R"•"S")") = $L(R)L(S)$, $R, S \in ER$
5. L ("("R"+"S")") = $L(R) \cup L(S)$, $R, S \in ER$
6. L ("("R")*") = $L(R)^*$, $R \in ER$

Para calcular el significado de una ER en particular, se le aplica una función L, y las ecuaciones anteriores se aplican repetidamente hasta que desaparece el símbolo L.

Ejemplo: El significado de la ER *"(((a + b))*• a)"* se calcula de la manera siguiente:

L (“(((a + b))• a)”)*	*= L (“((a + b))*”) L (“a”)*	(usando 4)
	= L (“(a + b)”) {a}*	(por 6 y 3)
	= (L (“a”) ∪ L (“b”)) {a}*	(aplicando 5)
	= ({a} ∪ {b}) {a}*	(usando 3)
	= {a, b}{a}*	(simplificando)

Este es el lenguaje de las palabras sobre *{a, b}* que terminan en *a*.



Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender que son las expresiones regulares:

María Alicia Piñeiro. (27 de junio de 2020). *Unidad 10: Expresiones Regulares.* [Archivo de Vídeo]. Youtube. https://youtu.be/TH_i2Z3KLsk

10.3 Metodología de diseño de las expresiones regulares

Al tratar de encontrar la ER para un lenguaje en particular, cuanto más complejo es el lenguaje, más difícil es encontrar la ER, usando pura intuición, por supuesto. En tales casos, puede ser práctico trabajar de manera organizada. Las técnicas que funcionan en muchos casos implican primero determinar la estructura y luego dejar algunos "huecos" para arreglar. Estos huecos, que llamamos *contextos*, son también el lenguaje en el que debe existir la ER.

Ejemplo: Obtener una ER para el lenguaje en el alfabeto {a, b, c} en que las palabras contienen exactamente una vez dos b contiguas. Por ejemplo, las palabras aabb, babba, pertenecen al lenguaje, pero no aaba, abbba ni bbabb. Para resolver este problema, expresamos primero la estructura de la ER de la manera siguiente:

Ejemplo: obtengamos una ER para un lenguaje en el alfabeto *{a, b, c}* donde la palabra contiene solo dos *b* consecutivas a la vez. Por ejemplo, las palabras *aabb*, *babba* pertenecen al lenguaje, pero no *aaba*, *abbba* o *bbabb*. Para resolver este problema, primero expresamos la estructura ER de la siguiente manera:

< primer contexto > *bb* < segundo contexto >

Observemos que en esta expresión aparece directamente la *bb's* que debería ser la ER, rodeada de otras dos ER, <primer contexto> y <segundo contexto>. El problema ahora es determinar qué ER corresponde a < primer contexto > y < segundo contexto >.

El lenguaje de <primer contexto> comprende a las palabras que no tienen *bb* y además no terminan en *b*. Esto es equivalente a decir que toda *b* está seguida de una *a* o una *c*. Esto quiere decir que la ER de este contexto va ser de la forma: *(...b (a + c)...)** donde los detalles que faltan están representados por los *"..."*. Lo que falta por considerar es que puede haber cualquier cantidad de *a's* o *c's* en el <primer contexto>, por lo que dicho contexto queda como:

$$(b\ (a + c) + a + c)^*$$

Similarmente se puede obtener la expresión para < segundo contexto >, que es

$$(a + c + (a + c)\ b)^*$$

Por lo que finalmente la ER del problema es:

$$(b\ (a + c) + a + c)^*\ bb\ (a + c + (a + c)\ b)^*$$

10.4 Equivalencias de las expresiones regulares

Las expresiones regulares no representan en forma única a un lenguaje (esto es, la función $L: ER \rightarrow 2^{\Sigma^*}$ descrita anteriormente no es inyectiva). Esto quiere decir que puede haber varias ER para un mismo lenguaje, lo cual desde luego no es conveniente, pues al ver dos ER distintas no podemos aun estar seguros de que representan dos lenguajes distintos. Por ejemplo, las ER *(a + b)** y *(a*b*)** representan el mismo lenguaje.

A diferencia de los autómatas finitos deterministas, no existe un procedimiento algorítmico que compare directamente dos ER; la comparación debe hacerse convirtiendo a AFD. Sin embargo, en algunos casos es útil utilizar ecuaciones de equivalencia entre las ER, que son de la expresión ER1 = ER2, lo que significa que el lenguaje de ER1 es el mismo que el de ER2, conteniendo las mismas palabras.

Por ejemplo, la equivalencia $R + S = S + R$ implica que la suma de expresiones regulares es conmutativa, por lo que, si tenemos dos ER particulares, como a^* y b^*ab, entonces la ER $a^* + b^*ab$ será equivalente a ER $b^* + ab^*$, ambos representando la misma palabra. La equivalencia $R + S = S + R$ puede resultar bastante obvia ya que se basa directamente en la conmutatividad de la unión de conjuntos, pero hay otras como $(R^*S)^* = \Lambda + (RS)^*S$ que son menos intuitivas.

La Tabla 3 contiene los equivalentes de ER más importantes divididos en 9 grupos.

Tabla 3

Equivalencias de las expresiones regulares más importantes

Grupo	Equivalencia
1	$R + S = S + R$ $(R + S) + T = R + (S + T)$ $R + \Phi = \Phi + R = R$ $R + R = R$
2	$R \bullet \Lambda = \Lambda \bullet R = R$ $R \bullet \Phi = \Phi \bullet R = \Phi$ $(R \bullet S) \bullet T = R \bullet (S \bullet T)$
3	$R \bullet (S + T) = R \bullet S + R \bullet T$ $(S + T) \bullet R = S \bullet R + T \bullet R$
4	$R^* = R^* \bullet R^* = (R^*)^* = (\Lambda + R)^*$ $\Phi^* = \Lambda^* = \varepsilon$
5	$R^* = \Lambda + RR^*$
6	$(R + S)^* = (R^* + S^*)^* = (R^*S^*)^* = (R^*S)^*R^* = R^*(SR^*)^* \neq R^* + S^*$
7	$R^*R = RR^*$ $R(SR)^* = (RS)^*R$
8	$(R^*S)^* = \Lambda + (R + S)^*S$ $(RS^*)^* = \Lambda + R(R + S)^*$
9	$R = SR + T$ *sí y sólo si* $R = S^*T$ $R = RS + T$ *sí y sólo si* $R = TS^*$

Fuente: *Nota.* Elaboración propia.

Estas equivalencias organizadas en 9 grupos se pueden utilizar para comprobar si dos ER representan el mismo lenguaje. Una técnica para comprobar que *P* = *Q* (donde P, Q ∈ ER) es construir la serie equivalente *P* = *R1* = *R2* =... = *Rn* = *Q*, utilizando las equivalencias de la Tabla 3.

Ejemplo: Verificar que las ER (ab + a)*a y a (ba + a)* son equivalentes, usando las equivalencias presentadas arriba.

Ejemplo: utilice las equivalencias de las ER para confirmar que las ER, *(ab + a)*a* y *a (ba + a)** son equivalentes.

*(ab + a)*a*	= *(a + ab)*a*	(por 1)
	= *(a*ab)*a*a*	(por 6)
	= *([a*a] b)* [a*a]*	(agrupamos términos)
	= *[a*a] (b [a*a])**	(por 7)
	= *aa* (baa*)**	(aplicando 7 a los términos entre corchetes)
	= *[a] [a* (baa*)*]*	(agrupando)
	= *a (a + ba)**	(por 6)
	= *a (ba + a)**	(por 1)

10.5 Equivalencia de expresiones regulares y autómatas finitos

El uso de las equivalencias para verificar que dos ER denotan el mismo lenguaje en general no es muy práctico, pues en muchos casos es difícil ver intuitivamente qué equivalencia conviene aplicar, además de que hay el riesgo de que al aplicarla nos alejemos de la solución en vez de acercarnos a ella. Para empeorar la situación, no hay metodologías generales que nos ayuden a diseñar estas pruebas. Es por esto por lo que normalmente probaremos la equivalencia de dos ER usando el procedimiento de conversión a un autómata finito. Por el teorema de Kleene, comprobaremos que para cualquier expresión regular existe un autómata finito equivalente.

Concepto

Crearemos algunas condiciones útiles para convertir expresiones regulares en máquinas de estados finitos y viceversa. El objetivo es hacer una transición gradual de una ER a un AF. Para pasar gradualmente de ER a AFN, la ER y el AFN deben usar un lenguaje común.

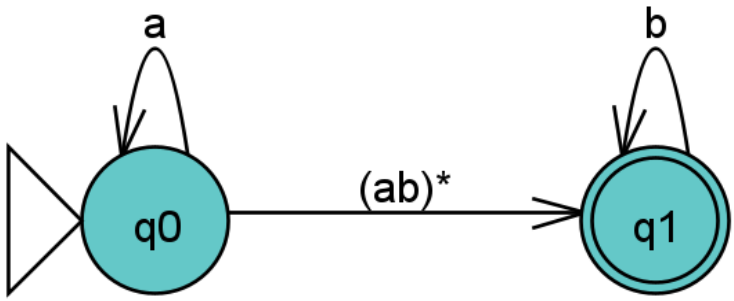
Para ello se usan las *gráficas de transición* (GT). Estas son AF en que las etiquetas de las flechas tienen expresiones regulares, en lugar de palabras. La Figura 3 muestra un ejemplo de GT.

Concepto

"Las gráficas de transición (GT) son quíntuplos M = (∑, K, δ , q1 , F) donde δ ∈K x ER x K" (Amaya, 2015, p. 52).

Figura 3

Ejemplo de una GT



Fuente: *Nota.* Elaboración propia.

En el ejemplo de la GT, se observa que se aceptan palabras que tienen primero una sucesión de *a's*, luego repeticiones de *ab*, y finalmente repeticiones de *b's*. La función está dada por:

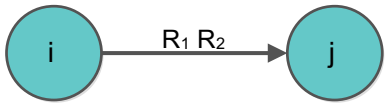
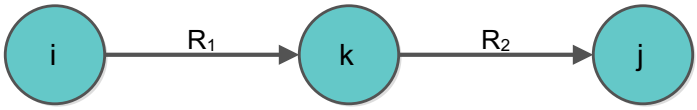
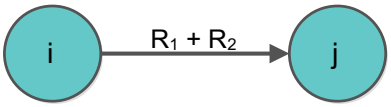
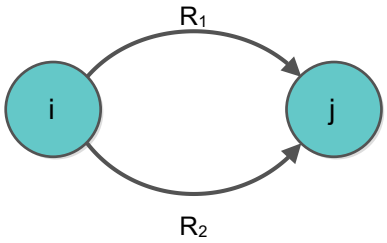
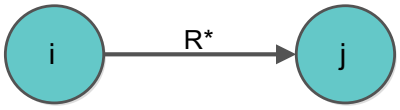
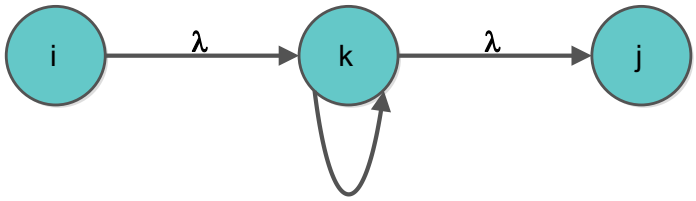
({q0, q1}, {a, b}, {(q0, a, q0), (q0, (ab), q1), (q1, b, q1)}, q0, {q1})*

Los AFN son un subconjunto adecuado de las GT, porque las palabras en las etiquetas del AFN se pueden tratar como sus propias expresiones regulares.

La Tabla 4 muestra un proceso de eliminación gradual de los operadores de las ER en donde *Gn* ∈ *AF* con las operaciones necesarias. Para eliminar los operadores de las ER en *Gi*, reemplazamos algunas transformaciones con otras hasta que no se puedan usar más reemplazos.

Tabla 4

Eliminación de operadores para convertir de ER a AF

Reemplazar	Por
	 Una transición sin alteraciones
 Dos opciones	 O es R_1 o es R_2
 Conjunto universal de Kleene	 Muchas, una o ninguna repetición

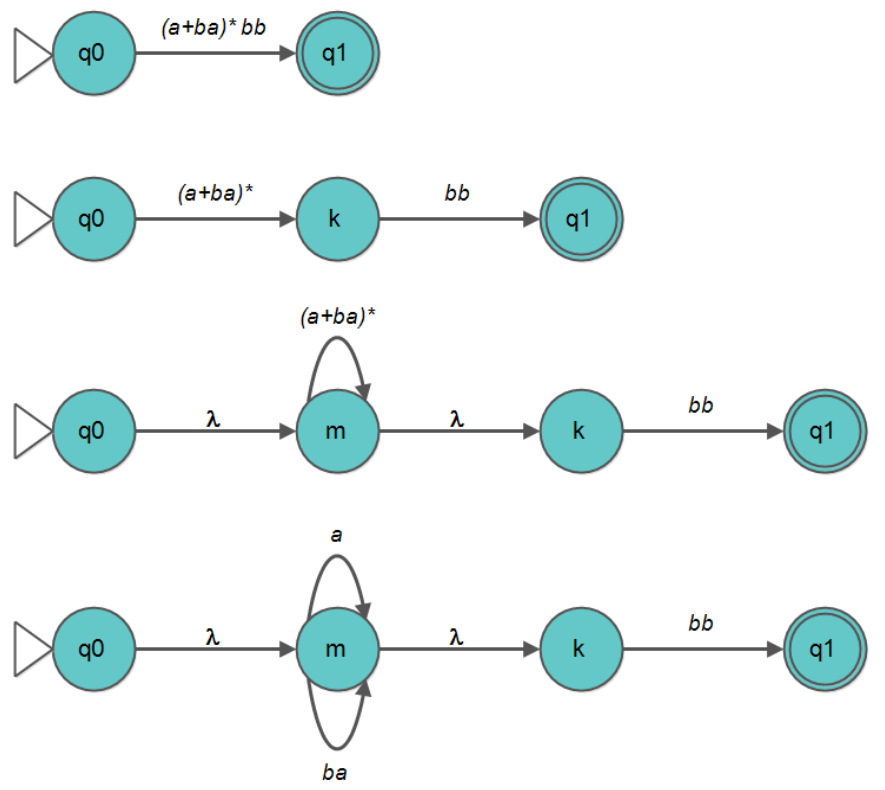
Fuente: *Nota.* Elaboración propia, a partir de Amaya (2015).

Por ejemplo, encontremos el AFN a partir de la *ER = (a + ba)* bb*.

El procedimiento consiste en aplicar las transformaciones mostradas en la Tabla 3. Las transformaciones se muestran en la Figura 4.

Figura 4

Trasformación de una ER a un AF

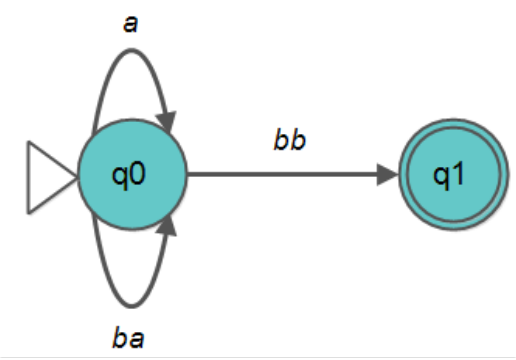


Fuente: *Nota.* Elaboración propia, a partir de Brena (2003)

Finalmente, en autómata se construye eliminando estados redundantes o innecesarios. En la Figura 5 se presenta el AFN a partir de la ER dada.

Figura 5

AF equivalente a la ER= (a + ba) bb*



Fuente: *Nota.* Elaboración propia, a partir de Brena (2003)

Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender las transformaciones de una expresión regular a un autómata finito:

Juancar Molinero. (2 de abril de 2019). *Expresiones regulares y autómatas finitos: problema de etiquetas.* [Archivo de Vídeo]. Youtube. https://youtu.be/VamCxfEO3Ak

Videos de elaboración propia

Clase 11| Gramáticas regulares

11. Gramáticas regulares

En esta clase veremos otra forma común de representar los lenguajes, además de las expresiones regulares y lo autómatas finitos, que ya hemos visto, también son utilizadas las gramáticas regulares que están asociadas a los lenguajes regulares.

11.1 Gramáticas formales

La representación de los lenguajes regulares se fundamenta en la noción de gramática formal. Intuitivamente, una gramática es un conjunto de reglas para formar correctamente las frases de un lenguaje; así tenemos la gramática del español, del francés, etc. La formalización que presentaremos de la noción de gramática es debida a Avram Noam Chomsky, y está basada en las llamadas reglas gramaticales.



Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender las Gramáticas Formales:

María Alicia Piñeiro. (27 de junio de 2020). *Unidad 10: Gramáticas Formales.* [Archivo de Vídeo]. Youtube. https://youtu.be/ZBs4Yv90jxc

11.2 Regla gramatical

Las reglas son expresiones de la forma $\alpha \rightarrow \beta$, donde tanto α como β son secuencias o cadenas de símbolos donde pueden aparecer tantos elementos del alfabeto Σ como nuevos símbolos llamados *variables*. Los símbolos denominados *no variables* son *constantes*. Por ejemplo, una posible regla gramatical es $X \rightarrow aX$. Aplicando la regla $\alpha \rightarrow \beta$ a la palabra $u\alpha v$ se obtiene la palabra $u\beta v$. Por lo tanto, las reglas gramaticales pueden considerarse como reglas de sustitución.

Por ejemplo, si tenemos una cadena de símbolos *bbXa*, podemos aplicarle la regla $X \rightarrow aX$, lo que da como resultado una nueva cadena *bbaXa*.

11.3 Gramáticas regulares

Estudiaremos las gramáticas con reglas de la forma $A \rightarrow aB$ o $A \rightarrow a$, donde *"A"* y *"B"* son variables y *"a"* es un símbolo terminal. Estas gramáticas se llaman regulares, por ejemplo, la gramática debe tener las siguientes reglas:

1) $S \rightarrow aA$
2) $S \rightarrow bA$
3) $A \rightarrow aB$
4) $A \rightarrow bB$
5) $A \rightarrow a$
6) $B \rightarrow aA$
7) $B \rightarrow bA$

La idea de aplicar una gramática es comenzar con una variable llamada símbolo inicial y aplicar las reglas de la gramática repetidamente hasta que no haya más variables en la palabra. La palabra resultante es *generada* por la gramática, o de manera equivalente, se dice que la palabra resultante es parte del lenguaje de esa gramática.

Por ejemplo, en la gramática que acabamos de presentar, si consideramos las variables S (que se convierte en el símbolo inicial), *A* y *B*, y las constantes *a* y *b*, podemos partir de *S* y producir *bA*. (segunda regla), después de bA podemos ir a *ba* (por la quinta regla). Dado que *ba* solo tiene constantes, podemos concluir que la palabra *"ba"* es parte del lenguaje generado por la gramática dada. De hecho, el lenguaje generado por esta gramática es el lenguaje de las palabras pares en los lenguajes *{a, b}* que terminan en *a*.



Para Amaya (2015) una gramática regular es un cuádruplo (V, Σ, R, S) en donde: V es un alfabeto de variables, Σ es un alfabeto de constantes, R el conjunto de reglas, es un subconjunto finito de $V \times (\Sigma V \cup \Sigma)$ y S es el símbolo inicial, es un elemento de V. (p. 67)

La gramática se representaría formalmente como:

({S, A, B}, {a, b}, {(S, aA), (S, bA), (A, aB), (A, bB), (A, a), (B, aA), (B, bA)}, S

Las reglas no suelen escribirse como pares ordenados *(A, aB)*, sino como $A \rightarrow aB$; se trata solo de la facilidad de notación. La aplicación de la gramática se formaliza mediante los siguientes términos:

- Una cadena uXv deriva en un paso una cadena uαv, escrito como uXv ⇒ uαv, si hay una regla X → α ∈ R en la gramática.
- Una cadena w ∈ Σ* (esto es, formada exclusivamente por constantes) es derivable a partir de una gramática G si existe una secuencia de pasos de derivación S ⇒ α1 ⇒ α2 ⇒... ⇒ w.
- A una secuencia de pasos de derivación le llamamos simplemente derivación.
- Dicho de otra manera, una palabra w ∈ Σ* es derivable a partir de G si sólo si S ⇒* w, donde ⇒* denota la cerradura reflexiva y transitiva de ⇒.
- La cadena *uXv* deriva la cadena *uαv* en un solo paso, que se escribe en la forma $uXv \Rightarrow u\alpha v$ si la gramática tiene la regla $X \rightarrow \alpha \in R$.
- La cadena $w \in \Sigma^*$ (es decir, consiste exclusivamente en constantes) es derivable de la gramática G si tiene una serie de pasos de derivación $S \Rightarrow \alpha 1 \Rightarrow \alpha 2 \Rightarrow ... \Rightarrow w.$
- Una secuencia de secuencia de pasos se denomina simplemente derivación.
- En otras palabras, una palabra $w \in \Sigma^*$ es derivable de G si y sólo si $S \Rightarrow^* w$, donde $\Rightarrow^*$ significa la cerradura reflexiva y transitiva de la palabra ⇒.

11.4 Lenguaje generado por una gramática

El lenguaje generado *L(G)* de una gramática G es igual al conjunto de palabras derivables de su símbolo inicial. Esto significa que $L(G) = \{w \in \Sigma^* / S \Rightarrow^* w\}$.

A menudo es fácil mostrar que una palabra dada *w* es derivable del símbolo inicial *S*; por ejemplo, en la gramática anterior se puede demostrar que $S \Rightarrow ... \Rightarrow bababa$ (es decir, la palabra *bababa* se puede derivar del símbolo inicial *S*, por lo que bababa $\in L(G)$.

Ejemplo: Estudiemos una gramática que cree un lenguaje para palabras *{a, b}* que contienen la subcadena *"bb"*, como *abb*, *abba*, etc. Usamos variables de la misma manera que los estados en un AF, es decir, como memoria para "recordar" estados, por lo que tenemos las siguientes variables:

- *A*, recuerda que aún no se ha producido ninguna *b*.
- *B*, recuerda que ha ocurrido una *b*.

- *C*, recuerda que las dos *b´s* ya han sido producidas.

Ahora podemos proponer reglas, preguntándonos a qué situación se llega al producir una “a” o “b”. Por ejemplo, a partir de A, si se produce una “a” se debe llegar a la misma A, pero si llega una “b” se llegará a la variable B. Con estas ideas se proponen las siguientes reglas:

Ahora podemos encontrar reglas preguntándonos qué situación surgirá si producimos una *"a"* o una *"b"*. Por ejemplo, a partir de *A*, cuando produce una *"a"*, debe llegar a la misma *A*, pero cuando llega una *"b"*, llega a la variable *B*. Estas ideas están relacionadas con las siguientes reglas:

1) $A \rightarrow aA$
2) $A \rightarrow bB$
3) $B \rightarrow aA$
4) $B \rightarrow bC$
5) $C \rightarrow aC$
6) $C \rightarrow bC$

Finalmente, se requiere al menos una regla que no produzca variables a la derecha para completar la creación de solo las palabras hechas de constantes. Tal regla aún no se encuentra en esta gramática. Como las palabras tienen *bb*, pensamos que la regla adicional podría ser $C \rightarrow a$ y también $C \rightarrow b$. En efecto, con tales reglas podemos producir, por ejemplo, la palabra "abba" con la siguiente derivación:

$A \Rightarrow aA \Rightarrow abB \Rightarrow abbC \Rightarrow abba$

No obstante, podemos verificar que las reglas dadas no pueden producir la palabra *"abb"* perteneciente al lenguaje. Necesitamos una regla más $B \rightarrow b$ para completar nuestra gramática. Al diseñar gramáticas regulares, podemos cometer los mismos errores que en los AF, es decir, son incorrectos (produce palabras que no deberían estar allí) o incompletos (no puede producir palabras que pertenecen al lenguaje) o ambos. No exploraremos métodos específicos para diseñar gramáticas comunes; En cambio, exploraremos métodos que facilitan la conversión de gramáticas regulares a un AF y viceversa.

11.5 Autómatas finitos y gramáticas regulares

De manera similar a la clase anterior sobre la equivalencia entre las *ER* y los *AF*, aquí se explica la equivalencia de la gramática y los lenguajes regulares y, por lo tanto, los autómatas finitos. Este resultado se confirma mediante la siguiente afirmación.

La clase de lenguajes generados por cualquier gramática regular es exactamente la clase de los lenguajes regulares.

La demostración de esta afirmación consiste en proporcionar un procedimiento para construir un autómata finito a partir de la gramática dada y viceversa.

Este procedimiento es sencillo y de manera directa, consiste en asociar los símbolos no finales (variables) de la gramática con *estados* del autómata. Así, para cada regla gramatical $A \rightarrow bC$, el autómata tiene una transición *(A, b, C)*. Sin embargo, el caso de las reglas $A \rightarrow b$ aún está pendiente. En estos casos, tenemos transiciones *(A, b, Z)*, donde *Z* es un nuevo estado para el que no hay un no terminal asociado; *Z* es el único estado final del autómata.

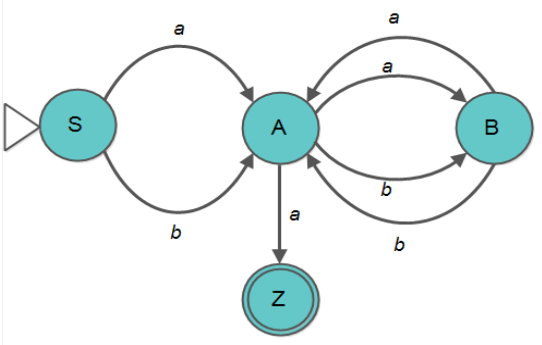
Ejemplo: Obtener un autómata finito para la gramática regular G siguiente:

1) $S \rightarrow aA$
2) $S \rightarrow bA$
3) $A \rightarrow aB$
4) $A \rightarrow bB$
5) $A \rightarrow a$
6) $B \rightarrow aA$
7) $B \rightarrow bA$

Dicho autómata se muestra en la Figura 6.

Figura 6

AF equivalente de la gramática regular del ejemplo



Fuente: *Nota.* Elaboración propia.

Conversión de un AFD a gramática regular. De manera similar, es fácil obtener la gramática regular correspondiente *(K, Σ, δ, s, F)* de DFA.

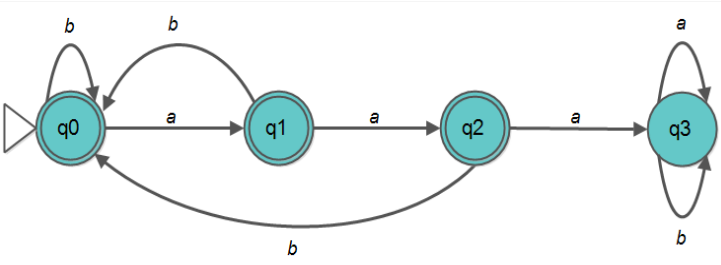
Para cada transición de la forma $((p, \sigma), q) \in \delta$, existirá en la gramática la regla $Xp \rightarrow \sigma Xq$, donde Xi es la variable gramatical correspondiente al estado i del AFD. Sin embargo, se debe revisar cómo obtener reglas de la forma $Xp \rightarrow \sigma$ que permitan la derivación.

Entendemos que la aplicación de las reglas debe corresponder al uso del último carácter de una palabra aceptado en el AFD. Ahora bien, al final de una palabra aceptada en el AFD, estamos necesariamente en el *estado final*. De esto concluimos que para cada transición $((p, \sigma), q)$, donde $q \in F$, la gramática debe contener una regla adicional $Xp \rightarrow \sigma$ además de la regla mencionada $Xp \rightarrow \sigma Xq$.

Ejemplo: Para el AFD de la figura 7, encontrar la gramática regular equivalente.

Figura 7

AFD al que se le encontrara la gramática regular equivalente



Fuente: *Nota.* Elaboración propia.

La gramática regular correspondiente es la siguiente:

1) $Q0 \rightarrow aQ1$

2) $Q0 \rightarrow bQ0$

3) $Q1 \rightarrow aQ2$

4) $Q1 \rightarrow bQ0$

5) $Q2 \rightarrow aQ3$

6) $Q2 \rightarrow bQ0$

7) $Q3 \rightarrow aQ3$

8) $Q3 \rightarrow bQ3$

9) $Q0 \rightarrow a$

10) $Q0 \rightarrow b$

11) $Q1 \rightarrow a$

12) $Q1 \rightarrow b$

13) $Q2 \rightarrow b$

Tenga en cuenta que, en este ejemplo, el AFD acepta la palabra vacía ε, mientras que la GR no puede generarla. De hecho, ningún GR puede producir ε. En tales casos, nos conformamos con generar un lenguaje igual al aceptado por el AFD, excepto por la palabra vacía.



Enlaces a recursos externos

Te invito a ver el siguiente video, para comprender las equivalencias entre los autómatas finitos y las gramáticas regulares:

Manuel Soto Romero. (5 de abril de 2021). *Lección 10: Equivalencia entre Autómatas Finitos y Gramáticas Regulares (UACM - TC).* [Archivo de Vídeo]. Youtube. https://youtu.be/8q3VgpcjGb0



Videos de elaboración propia

Clase 12| Autómatas, complejidad y computabilidad

12. Autómatas, complejidad y computabilidad

En teoría de los autómatas, teoría de complejidad y teoría de la computabilidad, comprender las limitaciones de las computadoras y los algoritmos ayuda a mejorar la forma en que pensamos y justifica diferentes estrategias. Cuando nos conducen a un problema indecidible en el que construir un algoritmo no siempre nos conduce a una respuesta exacta de “si” o “no”.

12.1 Teoría de los autómatas

La teoría de los autómatas se ocupa de las definiciones y propiedades de los modelos computacionales matemáticos que se aplican en diversas áreas de la informática. Siendo uno de estos modelos los autómatas finitos utilizados en:

- Procesamiento de textos.
- Compiladores.
- Diseño de Hardware.

Otros paradigmas de la teoría de autómatas están íntimamente relacionados con la teoría de lenguajes formales. En este contexto, los autómatas se utilizan como representaciones finitas de lenguajes formales, los autómatas juegan un papel importante en la teoría computacional, usadas en:

- Lenguajes de programación.
- Inteligencia Artificial.

12.2 Teoría de la complejidad

La teoría de la complejidad computacional es parte de la teoría informática que descubre los recursos necesarios en los cálculos para resolver. Refiriéndonos a cálculo complicados y difíciles. En este contexto, podemos: determinar la complejidad computacional como un número de recursos necesarios para realizar los cálculos. Por lo tanto, los cálculos complejos requerirán más recursos, comparado con uno de menor complejidad. Recursos compartidos tiempo de aprendizaje (recuento de pasos

implementar el algoritmo de resolución de problemas accidente) y espacio (la cantidad de memoria utilizada) proporcionado para la solución de problemas.

Por ejemplo. Si el cálculo requiere más tiempo que otro tiempo, decimos que es más complicado y llamamos complejidad del tiempo. Por otro lado, si el cálculo requiere más espacio de almacenamiento ¿Cuál más decimos que es más complejo, inclusivo? En este caso estamos hablando de complejidad espacial. esto es Aparentemente el tiempo que tardó en completarse cálculos en una computadora antigua de lo que se requiere para hacer el mismo cálculo con un computador moderno.

12.3 Complejidad computacional

La complejidad computacional comprueba el orden de complejidad de los algoritmos que resuelven un problema solucionable. Siendo esta la razón por considerar dos tipos de recursos computacionales necesarios para resolverlos los problemas:

- **Tiempo:** Es el número de pasos básicos del algoritmo para resolver el problema.
- **Espacio:** La cantidad de memoria utilizada para la resolución de problemas.

La complejidad del algoritmo se expresa como función del tamaño de entrada de un algoritmo, refiriéndose a la tasa de crecimiento de recursos.

- Ratio del Tiempo de ejecución (Temporal): T(n).
- Ratio del espacio de almacenamiento necesario (Espacial): S(n)

12.4 Teoría de la computabilidad

La teoría de la computabilidad se conoce como la teoría de la recursividad, esta es una de las cuatro partes que componen la lógica matemática. Además, la teoría calcula en conjunto con la teoría de la automatización de datos, el lenguaje y las máquinas, es esencial en la informática teórica, y ésta a su vez de la industria informática.

Esta teoría explora los límites de la resolución de problemas con algoritmos. Gran parte de la informática se dedica a la resolución algorítmica de problemas, por lo que el descubrimiento del dilema es una gran sorpresa. La teoría de la computabilidad es útil porque no trata de resolver estos problemas algorítmicamente, lo que ahorra tiempo y esfuerzo.

Concepto

Para Amaya (2015) La Teoría de la Computabilidad se ocupa de dividir el universo de todos los lenguajes sobre ∑, en aquellos lenguajes que pueden ser reconocidos por algoritmos efectivos y los que no. Ello conduce a las funciones no computables, es decir, a los problemas no resolubles. (p. 117)

12.5 Clasificación de problemas de decisión

Un problema de decisión es aquel al que se puede responder sí o no. Los problemas de decisión se clasifican según dos criterios:

- ***Teoría de la computabilidad***
  - Decidible (o resoluble algorítmicamente):
    - Si existe un proceso mecánico (Turing Machine, MT) que lo solucione.
    - Además, el MT debe detenerse para cada entrada.
  - Parcialmente decidible (reconocible):
    - Si existe un proceso mecánico (Turing Machine, MT) que lo solucione.
    - Además, MT debe detenerse para esas entradas como solución.
  - No decidible:
    - Si NO tiene solución.
- ***Teoría de la complejidad computacional***
  - Conjuntos de clases de complejidad (clase L, NL, P, P-completa, NP, NP-Completo, NP-Duro).

En este caso, nos centraremos únicamente en problemas de clase P, NP.y NP-completo. Sin embargo, el modelo debe ser considerado por esta diferencia. Teoría de la máquina de Turing. Además, debemos distinguir entre:

- **MT determinista:** para cada par (estado, símbolo) no hay más de un cambio a otro estado.
- **MT no desconocido:** hay al menos un par (estado, símbolo) con más de una transición a diferentes estados.

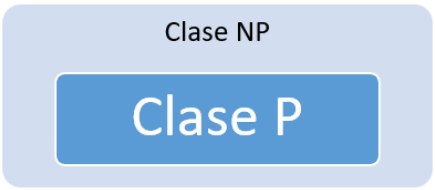
Teniendo en cuenta la teoría de la complejidad computacional, el problema de decisión puede ser:

- ***Clase P (Polynomial-time)***: contiene estos problemas de decisión que una máquina de Turing determinista puede resolver en tiempo polinomial. Los problemas de complejidad polinomial son tratables, es decir, en la práctica se pueden resolver en un tiempo razonable. Las tareas más comunes (ordenar, buscar) pertenecen a esta clase.
- ***Clase NP (Non-Deterministic Polynomial-time)***: contiene problemas de decisión que una máquina de Turing indefinida puede resolver en tiempo polinomial. Saber si $P = NP$ o $P \neq NP$ sigue siendo un problema abierto en el cálculo teórico.

Como toda *MTD* es un caso particular de una *MTND:* $P \subseteq NP$. En la Figura 8 muestra el diagrama de representación de problemas con la clase NP.

Figura 8

Diagrama de clase NP

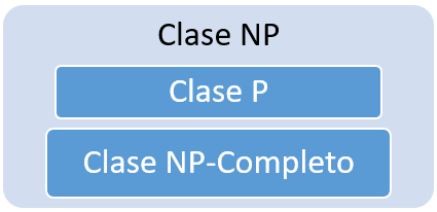


Fuente: *Nota.* Elaboración propia.

- ***Clase NP-Completo***: Un problema de decisión es NP-Completo sí y sólo si:
  - Es NP.
  - Todos los demás problemas NP le pertenecen en tiempo polinomial
- **Reducción de problemas:** Esta es una forma de transformar un problema en otro, de modo que la solución del segundo problema pueda usarse para resolver el primero. En la Figura 9 muestra la representación de una clase NP con su clase NP-Complemento.

Figura 9

Diagrama de clase NP que contiene una clase NP-Complemento



Fuente: *Nota.* Elaboración propia.

12.6 Complejidad espacial temporal

La complejidad espacial temporal es una medida del número de operaciones realizadas (entendido como una medida de tiempo que lleva realizar los cálculos). Entonces, para realizar un análisis, primero debe determinar el tamaño del problema. Luego, se elige una unidad para determinar la complejidad del espacio y una unidad para determinar la complejidad del tiempo. Sin embargo, a veces no se posee criterios claros para seleccionar estas unidades y la capacidad de determinar las acciones apropiadas.

El modelo de máquina de Turing facilita la definición de todos los conceptos anteriores:

- El tamaño del problema depende de la longitud de la cadena de entrada.
- La complejidad espacial está relacionada con el número de celdas de una máquina de Turing.
- La complejidad del tiempo depende del número de movimientos del cabezal, suponiendo que cada movimiento se realizará en el momento especificado.



Para Jurado (2008) la máquina M con k pistas tiene una *complejidad espacial* S(n), si para cualquier entrada de longitud n, consulta como máximo S(n) casillas. También podemos decir que M es una Máquina de Turing espacialmente acotada por S(n), o que L(M) es un lenguaje con complejidad espacial S(n). (p. 138)

Un movimiento o transición puede cambiar de estado (o quedarse en el estado actual), escribir un símbolo (reemplaza el símbolo que existía o dejando el mismo) y mover la cabeza a la izquierda o derecha. En la Figura 10 muestra una Máquina de Turing para este caso.

Figura 10

Máquina de Turing acotado



B	B	B	X1	X2	Xi	Xn	B	B	.....

Fuente: *Nota.* Elaboración propia.

Concepto

Para Jurado (2008) en una Máquina de Turing; si, para cualquier entrada de longitud "n", "M" realiza como máximo T (n) movimientos, de cabezal, se dice que "M" es una Máquina de Turing acotada temporalmente por la función T(n). También se dice que L (M) es un lenguaje con *complejidad temporal* T (n). (p. 138)

Si para cada palabra de entrada de longitud "n", "M" toma como máximo T(n) movimientos antes de detenerse, entonces M es un Máquina de Turing con tiempo limitado o complejidad de tiempo T(n). En la Figura 11 se muestra la Máquina de Turing para este caso.

Figura 11

Máquina de Turing con tiempo limitado o complejidad de tiempo



…	B	B	X1	X2	Xi	Xn	B	B	.....

Fuente: *Nota.* Elaboración propia.

Ejemplo: Considérese el lenguaje L = {wcw_R / w $\in$ (0 + 1)*}

El lenguaje "L" tiene una complejidad n1 veces debido a la máquina de Turing M1 con dos cintas copia la entrada a la izquierda del carácter "c" a la otra cinta. Luego, cuando encuentra la letra "c", M1 mueve el lector de su otra cinta hacia la izquierda, lee la línea que acaba de copiar y sigue moviéndose al mismo tiempo.

Lector de cinta de entrada a la derecha. Entonces comparamos símbolo mientras ambos extremos se mueven simultáneamente. Si las características a comparar son la misma, y la fuerza de la palabra antes del símbolo "c" es la misma que la potencia desde la derecha, M1 se hace cargo de la cadena. Es fácil ver lo que está haciendo el M1 a lo sumo n1 se mueve si la entrada es "n" de largo. Hay otra máquina de Turing, M2, con una complejidad en log2 n tomando "L". Aplicaciones de M2 dos cintas para almacenar el contador binario. Primero, verifique la entrada contiene solo una "c" y el número de caracteres a la izquierda de la "c" es como número de caracteres a la derecha. Entonces las palabras de la derecha y de la izquierda son iguales compare cada carácter, use el contador para buscar el carácter la derecha. Si no están de acuerdo, M2 deja de aceptar. Si todos los caracteres coinciden, entonces M2 lo acepta.



Referencias citadas en UNIDAD #3

- Brena, R. (2003). *Autómatas y lenguajes, un enfoque de diseño*. ITESM Monterey.
- Amaya, C. (2015). *Autómatas y lenguajes formales*. Universidad de Colombia.
- Hernández Rodríguez, L. A., Jaramillo Valbuena, S., & Cardona Torres, S. A. (2010). *Practique la teoría de autómatas y lenguajes formales*. Ediciones Elizcom.
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2008). Introducción a la teoría de autómatas, lenguajes y computación. *Pearson Educación S.A.*
- Jurado, Elena. (2008). *Teorías de Autómatas y Lenguajes Formales*. Universidad de Extremadura.



Glosario de los términos citados en la UNIDAD #3

Lenguajes regulares Son aquellos lenguajes cuyas cadenas está formadas por la concatenación de símbolos, en las cuales no hay relación entre una parte de la cadena y otra parte de la cadena. Los lenguajes regulares son aquellos que pueden describirse mediante un autómata finito.

Lema de bombeo Establece que, si un lenguaje es regular, entonces toda cadena lo suficientemente larga del lenguaje tiene una subcadena no vacía que puede ser "bombeada", es decir, repetida cualquier número de veces siempre y cuando las cadenas resultantes pertenezcan también al lenguaje. Este hecho puede utilizarse para demostrar que muchos lenguajes no son regulares.

Expresión regular (ER) Es una notación normalizada para representar lenguajes regulares, es decir, lenguajes generados por gramáticas de tipo 3.

Gráficas de transición (GT) Son quíntuplos M = (Σ, K, δ , q1, F) donde $\delta \in$K x ER x K.

Gramática regular (GR) Es un cuádruplo (V, Σ, R, S) en donde: V es un alfabeto de variables, Σ es un alfabeto de constantes, R el conjunto de reglas, es un subconjunto finito de V × (ΣV ∪ Σ) y S es el símbolo inicial, es un elemento de V.

Teoría de la Computabilidad Se ocupa de dividir el universo de todos los lenguajes sobre Σ, en aquellos lenguajes que pueden ser reconocidos por algoritmos efectivos y los que no. Ello conduce a las funciones no computables, es decir, a los problemas no resolubles.

Complejidad espacial

La máquina M con k pistas tiene una complejidad espacial S(n), si para cualquier entrada de longitud n, consulta como máximo S(n) casillas. También podemos decir que M es una Máquina de Turing espacialmente acotada por S(n), o que L(M) es un lenguaje con complejidad espacial S(n).

Complejidad temporal

En una Máquina de Turing; si, para cualquier entrada de longitud "n", "M" realiza como máximo T (n) movimientos, de cabezal, se dice que "M" es una Máquina de Turing acotada temporalmente por la función T(n). También se dice que L (M) es un lenguaje con complejidad temporal T (n).